

Failure modes, guardrails, and best practices

AI coding tools are useful enough that we should use them.

AI coding tools are useful enough that we should use them.

They are unreliable enough that we need names for the common failure modes.

Failure modes

AI coding tools are useful enough that we should use them.

They are unreliable enough that we need names for the common failure modes.

We will move from easy-to-catch failures to harder failures:

AI coding tools are useful enough that we should use them.

They are unreliable enough that we need names for the common failure modes.

We will move from easy-to-catch failures to harder failures:

1. correctness failures

AI coding tools are useful enough that we should use them.

They are unreliable enough that we need names for the common failure modes.

We will move from easy-to-catch failures to harder failures:

1. correctness failures
2. testing failures

AI coding tools are useful enough that we should use them.

They are unreliable enough that we need names for the common failure modes.

We will move from easy-to-catch failures to harder failures:

1. correctness failures
2. testing failures
3. feasibility failures

AI coding tools are useful enough that we should use them.

They are unreliable enough that we need names for the common failure modes.

We will move from easy-to-catch failures to harder failures:

1. correctness failures
2. testing failures
3. feasibility failures
4. persistence failures

AI coding tools are useful enough that we should use them.

They are unreliable enough that we need names for the common failure modes.

We will move from easy-to-catch failures to harder failures:

1. correctness failures
2. testing failures
3. feasibility failures
4. persistence failures
5. scope failures

AI coding tools are useful enough that we should use them.

They are unreliable enough that we need names for the common failure modes.

We will move from easy-to-catch failures to harder failures:

1. correctness failures
2. testing failures
3. feasibility failures
4. persistence failures
5. scope failures
6. security failures

AI coding tools are useful enough that we should use them.

They are unreliable enough that we need names for the common failure modes.

We will move from easy-to-catch failures to harder failures:

1. correctness failures
2. testing failures
3. feasibility failures
4. persistence failures
5. scope failures
6. security failures
7. instruction violations

AI coding tools are useful enough that we should use them.

They are unreliable enough that we need names for the common failure modes.

We will move from easy-to-catch failures to harder failures:

1. correctness failures
2. testing failures
3. feasibility failures
4. persistence failures
5. scope failures
6. security failures
7. instruction violations
8. communication failures

These are failures where the code is directly wrong.

These are failures where the code is directly wrong.

Examples:

Correctness failures

These are failures where the code is directly wrong.

Examples:

- syntax errors

These are failures where the code is directly wrong.

Examples:

- syntax errors
- hallucinated APIs

These are failures where the code is directly wrong.

Examples:

- syntax errors
- hallucinated APIs
- outdated APIs

These are failures where the code is directly wrong.

Examples:

- syntax errors
- hallucinated APIs
- outdated APIs
- incorrect algorithm

These are failures where the code is directly wrong.

Examples:

- syntax errors
- hallucinated APIs
- outdated APIs
- incorrect algorithm
- correct algorithm, wrong edge-case behavior

These are failures where the code is directly wrong.

Examples:

- syntax errors
- hallucinated APIs
- outdated APIs
- incorrect algorithm
- correct algorithm, wrong edge-case behavior

Correctness failures

These are failures where the code is directly wrong.

Examples:

- syntax errors
- hallucinated APIs
- outdated APIs
- incorrect algorithm
- correct algorithm, wrong edge-case behavior

These are the easiest failures to catch when you have tests.

Correctness failures

These are failures where the code is directly wrong.

Examples:

- syntax errors
- hallucinated APIs
- outdated APIs
- incorrect algorithm
- correct algorithm, wrong edge-case behavior

These are the easiest failures to catch when you have tests.

They are much harder to catch when you only read the code casually.

Worked example: hallucinated API

This looks plausible:

Worked example: hallucinated API

This looks plausible:

```
from scipy.special import beta_log_quad

def evidence_slab(n, k):
    return beta_log_quad(k + 1, n - k + 1)
```


Worked example: hallucinated API

This looks plausible:

```
from scipy.special import beta_log_quad

def evidence_slab(n, k):
    return beta_log_quad(k + 1, n - k + 1)
```

But `scipy.special.beta_log_quad` is not a real function.

Worked example: hallucinated API

This looks plausible:

```
from scipy.special import beta_log_quad
```

```
def evidence_slab(n, k):  
    return beta_log_quad(k + 1, n - k + 1)
```

But `scipy.special.beta_log_quad` is not a real function.

This failure is easy to catch:

Worked example: hallucinated API

This looks plausible:

```
from scipy.special import beta_log_quad
```

```
def evidence_slab(n, k):  
    return beta_log_quad(k + 1, n - k + 1)
```

But `scipy.special.beta_log_quad` is not a real function.

This failure is easy to catch:

```
ImportError: cannot import name 'beta_log_quad' from 'scipy.special'
```

LLMs often know older versions of libraries.

LLMs often know older versions of libraries.

They may generate code that:

LLMs often know older versions of libraries.

They may generate code that:

- works with a previous version

LLMs often know older versions of libraries.

They may generate code that:

- works with a previous version
- raises a deprecation warning now

LLMs often know older versions of libraries.

They may generate code that:

- works with a previous version
- raises a deprecation warning now
- will break in a future release

LLMs often know older versions of libraries.

They may generate code that:

- works with a previous version
- raises a deprecation warning now
- will break in a future release

LLMs often know older versions of libraries.

They may generate code that:

- works with a previous version
- raises a deprecation warning now
- will break in a future release

For Python test suites, one useful habit is:

LLMs often know older versions of libraries.

They may generate code that:

- works with a previous version
- raises a deprecation warning now
- will break in a future release

For Python test suites, one useful habit is:

```
python -W error::FutureWarning -m unittest discover -s tests
```

LLMs often know older versions of libraries.

They may generate code that:

- works with a previous version
- raises a deprecation warning now
- will break in a future release

For Python test suites, one useful habit is:

```
python -W error::FutureWarning -m unittest discover -s tests
```

Warnings are not always harmless.

AI tools are often good at writing tests.

Testing failures

AI tools are often good at writing tests.

They are also good at writing tests that pass.

Testing failures

AI tools are often good at writing tests.

They are also good at writing tests that pass.

Those are not the same thing.

Testing failures

AI tools are often good at writing tests.

They are also good at writing tests that pass.

Those are not the same thing.

Common testing failures:

Testing failures

AI tools are often good at writing tests.

They are also good at writing tests that pass.

Those are not the same thing.

Common testing failures:

- happy-path tests

Testing failures

AI tools are often good at writing tests.

They are also good at writing tests that pass.

Those are not the same thing.

Common testing failures:

- happy-path tests
- mock implementations that test the mock, not the code

Testing failures

AI tools are often good at writing tests.

They are also good at writing tests that pass.

Those are not the same thing.

Common testing failures:

- happy-path tests
- mock implementations that test the mock, not the code
- weak assertions

Testing failures

AI tools are often good at writing tests.

They are also good at writing tests that pass.

Those are not the same thing.

Common testing failures:

- happy-path tests
- mock implementations that test the mock, not the code
- weak assertions
- no checks for whether outputs changed

Testing failures

AI tools are often good at writing tests.

They are also good at writing tests that pass.

Those are not the same thing.

Common testing failures:

- happy-path tests
- mock implementations that test the mock, not the code
- weak assertions
- no checks for whether outputs changed
- exact equality for floating-point values

Testing failures

AI tools are often good at writing tests.

They are also good at writing tests that pass.

Those are not the same thing.

Common testing failures:

- happy-path tests
- mock implementations that test the mock, not the code
- weak assertions
- no checks for whether outputs changed
- exact equality for floating-point values
- coverage gaps

This test is almost a smoke test:

This test is almost a smoke test:

```
def test_evidence_slab_returns_float(self):  
    bf = BayesFactor(n=10, k=7)  
    self.assertIsInstance(bf.evidence_slab(), float)
```


This test is almost a smoke test:

```
def test_evidence_slab_returns_float(self):  
    bf = BayesFactor(n=10, k=7)  
    self.assertIsInstance(bf.evidence_slab(), float)
```

A wrong formula can pass this.

This test is almost a smoke test:

```
def test_evidence_slab_returns_float(self):  
    bf = BayesFactor(n=10, k=7)  
    self.assertIsInstance(bf.evidence_slab(), float)
```

A wrong formula can pass this.

It checks shape, not meaning.

For the uniform slab prior, the marginal likelihood has a known value:

For the uniform slab prior, the marginal likelihood has a known value:

$$P(k \mid n, H_{\text{slab}}) = \int_0^1 \binom{n}{k} \theta^k (1 - \theta)^{n-k} d\theta$$

For the uniform slab prior, the marginal likelihood has a known value:

$$P(k \mid n, H_{\text{slab}}) = \int_0^1 \binom{n}{k} \theta^k (1 - \theta)^{n-k} d\theta$$

For the uniform prior on $[0, 1]$, this simplifies to:

For the uniform slab prior, the marginal likelihood has a known value:

$$P(k \mid n, H_{\text{slab}}) = \int_0^1 \binom{n}{k} \theta^k (1 - \theta)^{n-k} d\theta$$

For the uniform prior on $[0, 1]$, this simplifies to:

$$P(k \mid n, H_{\text{slab}}) = \frac{1}{n+1}$$

```
def test_evidence_slab_matches_closed_form(self):  
    bf = BayesFactor(n=10, k=7)  
    self.assertAlmostEqual(bf.evidence_slab(), 1 / 11, places=10)
```

```
def test_evidence_slab_matches_closed_form(self):  
    bf = BayesFactor(n=10, k=7)  
    self.assertAlmostEqual(bf.evidence_slab(), 1 / 11, places=10)
```

This test can fail for the right reason.


```
def test_evidence_slab_matches_closed_form(self):  
    bf = BayesFactor(n=10, k=7)  
    self.assertAlmostEqual(bf.evidence_slab(), 1 / 11, places=10)
```

This test can fail for the right reason.

That is what you want.

Avoid exact equality for floating-point results:

Floating-point comparisons

Avoid exact equality for floating-point results:

```
self.assertEqual(bf.evidence_slab(), 1 / 11) # brittle
```

Avoid exact equality for floating-point results:

```
self.assertEqual(bf.evidence_slab(), 1 / 11) # brittle
```

Prefer tolerance:

Avoid exact equality for floating-point results:

```
self.assertEqual(bf.evidence_slab(), 1 / 11) # brittle
```

Prefer tolerance:

```
self.assertAlmostEqual(bf.evidence_slab(), 1 / 11, places=10)
```

Floating-point comparisons

Avoid exact equality for floating-point results:

```
self.assertEqual(bf.evidence_slab(), 1 / 11) # brittle
```

Prefer tolerance:

```
self.assertAlmostEqual(bf.evidence_slab(), 1 / 11, places=10)
```

The tolerance should be tight enough to catch real mistakes.

Sometimes the code is not the main problem, but the AI just proposes a wrong approach.

Sometimes the code is not the main problem, but the AI just proposes a wrong approach.

AI models can confidently say a plan is feasible when:

Sometimes the code is not the main problem, but the AI just proposes a wrong approach.

AI models can confidently say a plan is feasible when:

- the algorithm does not fit the hardware

Sometimes the code is not the main problem, but the AI just proposes a wrong approach.

AI models can confidently say a plan is feasible when:

- the algorithm does not fit the hardware
- the complexity will not scale

Sometimes the code is not the main problem, but the AI just proposes a wrong approach.

AI models can confidently say a plan is feasible when:

- the algorithm does not fit the hardware
- the complexity will not scale
- the necessary library capability does not exist

Sometimes the code is not the main problem, but the AI just proposes a wrong approach.

AI models can confidently say a plan is feasible when:

- the algorithm does not fit the hardware
- the complexity will not scale
- the necessary library capability does not exist
- the model has confused similar tools or environments

Sometimes the code is not the main problem, but the AI just proposes a wrong approach.

AI models can confidently say a plan is feasible when:

- the algorithm does not fit the hardware
- the complexity will not scale
- the necessary library capability does not exist
- the model has confused similar tools or environments

Sometimes the code is not the main problem, but the AI just proposes a wrong approach.

AI models can confidently say a plan is feasible when:

- the algorithm does not fit the hardware
- the complexity will not scale
- the necessary library capability does not exist
- the model has confused similar tools or environments

This is where domain expertise matters most.

Project: speed up a statistical workflow by making use of GPU processing.

Worked example: GPU acceleration

Project: speed up a statistical workflow by making use of GPU processing.

The model repeatedly treated the task as feasible.

Worked example: GPU acceleration

Project: speed up a statistical workflow by making use of GPU processing.

The model repeatedly treated the task as feasible.

After quite some debugging, realized the problem was architectural:

Worked example: GPU acceleration

Project: speed up a statistical workflow by making use of GPU processing.

The model repeatedly treated the task as feasible.

After quite some debugging, realized the problem was architectural:

- the relevant algorithm worked well on recent versions of CUDA

Worked example: GPU acceleration

Project: speed up a statistical workflow by making use of GPU processing.

The model repeatedly treated the task as feasible.

After quite some debugging, realized the problem was architectural:

- the relevant algorithm worked well on recent versions of CUDA
- lab had old GPUs with older drivers

Project: speed up a statistical workflow by making use of GPU processing.

The model repeatedly treated the task as feasible.

After quite some debugging, realized the problem was architectural:

- the relevant algorithm worked well on recent versions of CUDA
- lab had old GPUs with older drivers
- PyTorch operations could not reproduce the CUDA implementation efficiently

If the model keeps trying variations of the same failing strategy, treat that as evidence.

When the model loops

If the model keeps trying variations of the same failing strategy, treat that as evidence.

Possible interpretations:

If the model keeps trying variations of the same failing strategy, treat that as evidence.

Possible interpretations:

- the requirements are unclear

If the model keeps trying variations of the same failing strategy, treat that as evidence.

Possible interpretations:

- the requirements are unclear
- the tests are too weak or misleading

If the model keeps trying variations of the same failing strategy, treat that as evidence.

Possible interpretations:

- the requirements are unclear
- the tests are too weak or misleading
- the approach is infeasible

If the model keeps trying variations of the same failing strategy, treat that as evidence.

Possible interpretations:

- the requirements are unclear
- the tests are too weak or misleading
- the approach is infeasible
- the model lacks the domain knowledge to choose a new strategy

If the model keeps trying variations of the same failing strategy, treat that as evidence.

Possible interpretations:

- the requirements are unclear
- the tests are too weak or misleading
- the approach is infeasible
- the model lacks the domain knowledge to choose a new strategy

When the model loops

If the model keeps trying variations of the same failing strategy, treat that as evidence.

Possible interpretations:

- the requirements are unclear
- the tests are too weak or misleading
- the approach is infeasible
- the model lacks the domain knowledge to choose a new strategy

Do not reward infinite persistence, be prepared to stop, review, and reframe the challenge.

Two opposite problems:

Persistence failures

Two opposite problems:

Overpersistence

Two opposite problems:

Overpersistence

- infinite iteration loops

Two opposite problems:

Overpersistence

- infinite iteration loops
- whack-a-mole fixes

Two opposite problems:

Overpersistence

- infinite iteration loops
- whack-a-mole fixes
- retrying cosmetic variations of the same idea

Two opposite problems:

Overpersistence

- infinite iteration loops
- whack-a-mole fixes
- retrying cosmetic variations of the same idea

Two opposite problems:

Overpersistence

- infinite iteration loops
- whack-a-mole fixes
- retrying cosmetic variations of the same idea

Underpersistence

Persistence failures

Two opposite problems:

Overpersistence

- infinite iteration loops
- whack-a-mole fixes
- retrying cosmetic variations of the same idea

Underpersistence

- solving a simplified problem instead

Two opposite problems:

Overpersistence

- infinite iteration loops
- whack-a-mole fixes
- retrying cosmetic variations of the same idea

Underpersistence

- solving a simplified problem instead
- adding a workaround

Two opposite problems:

Overpersistence

- infinite iteration loops
- whack-a-mole fixes
- retrying cosmetic variations of the same idea

Underpersistence

- solving a simplified problem instead
- adding a workaround
- declaring success before the requirement is met

Two opposite problems:

Overpersistence

- infinite iteration loops
- whack-a-mole fixes
- retrying cosmetic variations of the same idea

Underpersistence

- solving a simplified problem instead
- adding a workaround
- declaring success before the requirement is met

Persistence failures

Two opposite problems:

Overpersistence

- infinite iteration loops
- whack-a-mole fixes
- retrying cosmetic variations of the same idea

Underpersistence

- solving a simplified problem instead
- adding a workaround
- declaring success before the requirement is met

Both require a human to notice.

AI coding tools often do more or less than asked.

AI coding tools often do more or less than asked.

Common forms:

AI coding tools often do more or less than asked.

Common forms:

- gold-plating

AI coding tools often do more or less than asked.

Common forms:

- gold-plating
- scope creep

AI coding tools often do more or less than asked.

Common forms:

- gold-plating
- scope creep
- premature abstraction

AI coding tools often do more or less than asked.

Common forms:

- gold-plating
- scope creep
- premature abstraction
- large class hierarchies for small problems

AI coding tools often do more or less than asked.

Common forms:

- gold-plating
- scope creep
- premature abstraction
- large class hierarchies for small problems
- premature declaration of success

AI coding tools often do more or less than asked.

Common forms:

- gold-plating
- scope creep
- premature abstraction
- large class hierarchies for small problems
- premature declaration of success

AI coding tools often do more or less than asked.

Common forms:

- gold-plating
- scope creep
- premature abstraction
- large class hierarchies for small problems
- premature declaration of success

It's best not to waste time iterating on these issues. Instead, write skills files that set clear acceptance criteria, and review the code.

Prompt:

Worked example: scope creep

Prompt:

Add a method to compute the log Bayes factor.

Worked example: scope creep

Prompt:

Add a method to compute the log Bayes factor.

Bad AI response:

Worked example: scope creep

Prompt:

Add a method to compute the log Bayes factor.

Bad AI response:

- adds `log_bayes_factor`

Worked example: scope creep

Prompt:

Add a method to compute the log Bayes factor.

Bad AI response:

- adds `log_bayes_factor`
- rewrites the constructor

Worked example: scope creep

Prompt:

Add a method to compute the log Bayes factor.

Bad AI response:

- adds `log_bayes_factor`
- rewrites the constructor
- changes `evidence_slab`

Worked example: scope creep

Prompt:

Add a method to compute the log Bayes factor.

Bad AI response:

- adds `log_bayes_factor`
- rewrites the constructor
- changes `evidence_slab`
- adds a plotting function

Worked example: scope creep

Prompt:

Add a method to compute the log Bayes factor.

Bad AI response:

- adds `log_bayes_factor`
- rewrites the constructor
- changes `evidence_slab`
- adds a plotting function
- replaces the test suite

Worked example: scope creep

Prompt:

Add a method to compute the log Bayes factor.

Bad AI response:

- adds `log_bayes_factor`
- rewrites the constructor
- changes `evidence_slab`
- adds a plotting function
- replaces the test suite

Worked example: scope creep

Prompt:

Add a method to compute the log Bayes factor.

Bad AI response:

- adds `log_bayes_factor`
- rewrites the constructor
- changes `evidence_slab`
- adds a plotting function
- replaces the test suite

The extra work is not harmless! You'd need additional tests, not an entirely new suite.

AI-generated code can introduce security problems.

AI-generated code can introduce security problems.

Examples:

AI-generated code can introduce security problems.

Examples:

- hard-coded credentials

AI-generated code can introduce security problems.

Examples:

- hard-coded credentials
- shell injection

AI-generated code can introduce security problems.

Examples:

- hard-coded credentials
- shell injection
- unsafe deserialization with `pickle`

AI-generated code can introduce security problems.

Examples:

- hard-coded credentials
- shell injection
- unsafe deserialization with `pickle`
- installing typo-squatted packages

AI-generated code can introduce security problems.

Examples:

- hard-coded credentials
- shell injection
- unsafe deserialization with `pickle`
- installing typo-squatted packages
- connecting agents to sensitive data

AI-generated code can introduce security problems.

Examples:

- hard-coded credentials
- shell injection
- unsafe deserialization with `pickle`
- installing typo-squatted packages
- connecting agents to sensitive data
- unsafe autonomous modes

AI-generated code can introduce security problems.

Examples:

- hard-coded credentials
- shell injection
- unsafe deserialization with `pickle`
- installing typo-squatted packages
- connecting agents to sensitive data
- unsafe autonomous modes

AI-generated code can introduce security problems.

Examples:

- hard-coded credentials
- shell injection
- unsafe deserialization with `pickle`
- installing typo-squatted packages
- connecting agents to sensitive data
- unsafe autonomous modes

Scientific code is not exempt from security.

Never do this:

Never do this:

```
API_KEY = "AIzaSy..."
```

Never do this:

```
API_KEY = "AIzaSy..."
```

Prefer:

Never do this:

```
API_KEY = "AIzaSy..."
```

Prefer:

```
API_KEY = os.environ["GEMINI_API_KEY"]
```

Never do this:

```
API_KEY = "AIzaSy..."
```

Prefer:

```
API_KEY = os.environ["GEMINI_API_KEY"]
```

And never commit `.env` files or local key files.

Agents may suggest installing packages. Make sure it hasn't been tricked!

Agents may suggest installing packages. Make sure it hasn't been tricked!

- `matplotlib`

Agents may suggest installing packages. Make sure it hasn't been tricked!

- `matplotlib`
- `requests`

Unsafe dependencies

Agents may suggest installing packages. Make sure it hasn't been tricked!

- `matplotlib`
- `requests`
- `numpy`

Agents may suggest installing packages. Make sure it hasn't been tricked!

- `matplotlib`
- `requests`
- `numpy`

Unsafe dependencies

Agents may suggest installing packages. Make sure it hasn't been tricked!

- `matplotlib`
- `requests`
- `numpy`

Before installing a new dependency, check that you know what it is, or check:

Agents may suggest installing packages. Make sure it hasn't been tricked!

- `matplotlib`
- `requests`
- `numpy`

Before installing a new dependency, check that you know what it is, or check:

- name

Agents may suggest installing packages. Make sure it hasn't been tricked!

- `matplotlib`
- `requests`
- `numpy`

Before installing a new dependency, check that you know what it is, or check:

- name
- maintainer / documentation

Unsafe dependencies

Agents may suggest installing packages. Make sure it hasn't been tricked!

- `matplotlib`
- `requests`
- `numpy`

Before installing a new dependency, check that you know what it is, or check:

- name
- maintainer / documentation
- whether the standard library or existing dependencies already solve the problem

Agents may ignore explicit instructions.

Instruction violations

Agents may ignore explicit instructions.

Examples:

Instruction violations

Agents may ignore explicit instructions.

Examples:

- uses `pytest` when asked for `unittest`

Instruction violations

Agents may ignore explicit instructions.

Examples:

- uses `pytest` when asked for `unittest`
- edits tests to pass implementation

Instruction violations

Agents may ignore explicit instructions.

Examples:

- uses `pytest` when asked for `unittest`
- edits tests to pass implementation
- makes one huge commit

Instruction violations

Agents may ignore explicit instructions.

Examples:

- uses `pytest` when asked for `unittest`
- edits tests to pass implementation
- makes one huge commit
- rewrites files outside the requested scope

Instruction violations

Agents may ignore explicit instructions.

Examples:

- uses **pytest** when asked for **unittest**
- edits tests to pass implementation
- makes one huge commit
- rewrites files outside the requested scope
- changes public APIs without being asked

Instruction violations

Agents may ignore explicit instructions.

Examples:

- uses **pytest** when asked for **unittest**
- edits tests to pass implementation
- makes one huge commit
- rewrites files outside the requested scope
- changes public APIs without being asked

Instruction violations

Agents may ignore explicit instructions.

Examples:

- uses **pytest** when asked for **unittest**
- edits tests to pass implementation
- makes one huge commit
- rewrites files outside the requested scope
- changes public APIs without being asked

This often happens when context is bloated or contradictory.

Instruction violations

Agents may ignore explicit instructions.

Examples:

- uses **pytest** when asked for **unittest**
- edits tests to pass implementation
- makes one huge commit
- rewrites files outside the requested scope
- changes public APIs without being asked

This often happens when context is bloated or contradictory.

Clear instructions help, but they are not a guarantee.

Instruction violations

Agents may ignore explicit instructions.

Examples:

- uses **pytest** when asked for **unittest**
- edits tests to pass implementation
- makes one huge commit
- rewrites files outside the requested scope
- changes public APIs without being asked

This often happens when context is bloated or contradictory.

Clear instructions help, but they are not a guarantee.

Keep your skills files tidy!

Models can misreport the state of the work.

Models can misreport the state of the work.

Common examples:

Models can misreport the state of the work.

Common examples:

- “All tests pass” when tests were not run

Models can misreport the state of the work.

Common examples:

- “All tests pass” when tests were not run
- “The bug is fixed” when only the example case works

Models can misreport the state of the work.

Common examples:

- “All tests pass” when tests were not run
- “The bug is fixed” when only the example case works
- confident diagnosis of the wrong cause

Models can misreport the state of the work.

Common examples:

- “All tests pass” when tests were not run
- “The bug is fixed” when only the example case works
- confident diagnosis of the wrong cause
- no expression of uncertainty

Models can misreport the state of the work.

Common examples:

- “All tests pass” when tests were not run
- “The bug is fixed” when only the example case works
- confident diagnosis of the wrong cause
- no expression of uncertainty

Models can misreport the state of the work.

Common examples:

- “All tests pass” when tests were not run
- “The bug is fixed” when only the example case works
- confident diagnosis of the wrong cause
- no expression of uncertainty

Ask for evidence:

Models can misreport the state of the work.

Common examples:

- “All tests pass” when tests were not run
- “The bug is fixed” when only the example case works
- confident diagnosis of the wrong cause
- no expression of uncertainty

Ask for evidence:

- command run

Models can misreport the state of the work.

Common examples:

- “All tests pass” when tests were not run
- “The bug is fixed” when only the example case works
- confident diagnosis of the wrong cause
- no expression of uncertainty

Ask for evidence:

- command run
- output observed

Models can misreport the state of the work.

Common examples:

- “All tests pass” when tests were not run
- “The bug is fixed” when only the example case works
- confident diagnosis of the wrong cause
- no expression of uncertainty

Ask for evidence:

- command run
- output observed
- files changed

Models can misreport the state of the work.

Common examples:

- “All tests pass” when tests were not run
- “The bug is fixed” when only the example case works
- confident diagnosis of the wrong cause
- no expression of uncertainty

Ask for evidence:

- command run
- output observed
- files changed
- tests added

AI-generated code often avoids obvious syntax problems.

AI-generated code often avoids obvious syntax problems.

The smells can be subtler:

AI-generated code often avoids obvious syntax problems.

The smells can be subtler:

- silent error swallowing

AI-generated code often avoids obvious syntax problems.

The smells can be subtler:

- silent error swallowing
- overly complex class hierarchies

AI-generated code often avoids obvious syntax problems.

The smells can be subtler:

- silent error swallowing
- overly complex class hierarchies
- leftovers from earlier attempts

AI-generated code often avoids obvious syntax problems.

The smells can be subtler:

- silent error swallowing
- overly complex class hierarchies
- leftovers from earlier attempts
- inappropriate pattern imitation

AI-generated code often avoids obvious syntax problems.

The smells can be subtler:

- silent error swallowing
- overly complex class hierarchies
- leftovers from earlier attempts
- inappropriate pattern imitation
- inconsistent style across files

AI-generated code often avoids obvious syntax problems.

The smells can be subtler:

- silent error swallowing
- overly complex class hierarchies
- leftovers from earlier attempts
- inappropriate pattern imitation
- inconsistent style across files
- docstrings that sound right but describe the wrong behavior

AI-generated code often avoids obvious syntax problems.

The smells can be subtler:

- silent error swallowing
- overly complex class hierarchies
- leftovers from earlier attempts
- inappropriate pattern imitation
- inconsistent style across files
- docstrings that sound right but describe the wrong behavior
- suspiciously broad exception handling

Bad:

Worked example: silent error swallowing

Bad:

```
def safe_bayes_factor(bf):  
    try:  
        return bf.bayes_factor()  
    except Exception:  
        return 1.0
```

Worked example: silent error swallowing

Bad:

```
def safe_bayes_factor(bf):  
    try:  
        return bf.bayes_factor()  
    except Exception:  
        return 1.0
```

This turns an error into neutral evidence.

```
def checked_bayes_factor(bf):  
    try:  
        return bf.bayes_factor()  
    except ZeroDivisionError as err:  
        raise ValueError("Bayes factor denominator was zero") from err
```

Three guardrails

An industry version of the same argument appears in CodeScene's discussion of AI coding guardrails.

Three guardrails

An industry version of the same argument appears in CodeScene's discussion of AI coding guardrails.

The useful triad:

Three guardrails

An industry version of the same argument appears in CodeScene's discussion of AI coding guardrails.

The useful triad:

1. Code quality

Three guardrails

An industry version of the same argument appears in CodeScene's discussion of AI coding guardrails.

The useful triad:

1. Code quality
2. Code familiarity

Three guardrails

An industry version of the same argument appears in CodeScene's discussion of AI coding guardrails.

The useful triad:

1. Code quality
2. Code familiarity
3. Test coverage

Three guardrails

An industry version of the same argument appears in CodeScene's discussion of AI coding guardrails.

The useful triad:

1. Code quality
2. Code familiarity
3. Test coverage

Three guardrails

An industry version of the same argument appears in CodeScene's discussion of AI coding guardrails.

The useful triad:

1. Code quality
2. Code familiarity
3. Test coverage

We do not need their tooling for this point.

An industry version of the same argument appears in CodeScene's discussion of AI coding guardrails.

The useful triad:

1. Code quality
2. Code familiarity
3. Test coverage

We do not need their tooling for this point.

The point is that faster code generation increases the need for reviewable, familiar, tested code.

AI-generated code should meet the same quality bar as human-generated code.

AI-generated code should meet the same quality bar as human-generated code.

Ask:

AI-generated code should meet the same quality bar as human-generated code.

Ask:

- Is it readable?

AI-generated code should meet the same quality bar as human-generated code.

Ask:

- Is it readable?
- Is it small enough to test?

AI-generated code should meet the same quality bar as human-generated code.

Ask:

- Is it readable?
- Is it small enough to test?
- Does it use names that reveal intent?

AI-generated code should meet the same quality bar as human-generated code.

Ask:

- Is it readable?
- Is it small enough to test?
- Does it use names that reveal intent?
- Does it follow the surrounding style?

AI-generated code should meet the same quality bar as human-generated code.

Ask:

- Is it readable?
- Is it small enough to test?
- Does it use names that reveal intent?
- Does it follow the surrounding style?
- Does it avoid speculative generality?

AI-generated code should meet the same quality bar as human-generated code.

Ask:

- Is it readable?
- Is it small enough to test?
- Does it use names that reveal intent?
- Does it follow the surrounding style?
- Does it avoid speculative generality?

AI-generated code should meet the same quality bar as human-generated code.

Ask:

- Is it readable?
- Is it small enough to test?
- Does it use names that reveal intent?
- Does it follow the surrounding style?
- Does it avoid speculative generality?

Fast bad code is still bad code.

If AI writes code you do not understand, you have created maintenance debt.

If AI writes code you do not understand, you have created maintenance debt.

You should be able to explain:

If AI writes code you do not understand, you have created maintenance debt.

You should be able to explain:

- what each public method does

If AI writes code you do not understand, you have created maintenance debt.

You should be able to explain:

- what each public method does
- why the algorithm is appropriate

If AI writes code you do not understand, you have created maintenance debt.

You should be able to explain:

- what each public method does
- why the algorithm is appropriate
- what assumptions it makes

If AI writes code you do not understand, you have created maintenance debt.

You should be able to explain:

- what each public method does
- why the algorithm is appropriate
- what assumptions it makes
- what its tests actually check

If AI writes code you do not understand, you have created maintenance debt.

You should be able to explain:

- what each public method does
- why the algorithm is appropriate
- what assumptions it makes
- what its tests actually check
- what failure modes remain

If AI writes code you do not understand, you have created maintenance debt.

You should be able to explain:

- what each public method does
- why the algorithm is appropriate
- what assumptions it makes
- what its tests actually check
- what failure modes remain

Guardrail 2: code familiarity

If AI writes code you do not understand, you have created maintenance debt.

You should be able to explain:

- what each public method does
- why the algorithm is appropriate
- what assumptions it makes
- what its tests actually check
- what failure modes remain

Understanding code is often the bottleneck, not typing it.

Tests are the executable specification.

Tests are the executable specification.

For AI-assisted coding, tests should include:

Tests are the executable specification.

For AI-assisted coding, tests should include:

- ordinary expected cases

Tests are the executable specification.

For AI-assisted coding, tests should include:

- ordinary expected cases
- edge cases

Tests are the executable specification.

For AI-assisted coding, tests should include:

- ordinary expected cases
- edge cases
- invalid input cases

Tests are the executable specification.

For AI-assisted coding, tests should include:

- ordinary expected cases
- edge cases
- invalid input cases
- regression tests for discovered bugs

Tests are the executable specification.

For AI-assisted coding, tests should include:

- ordinary expected cases
- edge cases
- invalid input cases
- regression tests for discovered bugs
- numerical tolerance choices

Tests are the executable specification.

For AI-assisted coding, tests should include:

- ordinary expected cases
- edge cases
- invalid input cases
- regression tests for discovered bugs
- numerical tolerance choices

Tests are the executable specification.

For AI-assisted coding, tests should include:

- ordinary expected cases
- edge cases
- invalid input cases
- regression tests for discovered bugs
- numerical tolerance choices

Coverage alone is not enough.

Guardrail 3: test coverage

Tests are the executable specification.

For AI-assisted coding, tests should include:

- ordinary expected cases
- edge cases
- invalid input cases
- regression tests for discovered bugs
- numerical tolerance choices

Coverage alone is not enough.

The assertions must mean something.

Tests and implementation should not be invented by the same unchecked process.

Tests and implementation should not be invented by the same unchecked process.

If a model writes the code and the tests, it may write tests that merely confirm its own assumptions.

Tests and implementation should not be invented by the same unchecked process.

If a model writes the code and the tests, it may write tests that merely confirm its own assumptions.

Humans must own at least one side of the ledger:

Tests and implementation should not be invented by the same unchecked process.

If a model writes the code and the tests, it may write tests that merely confirm its own assumptions.

Humans must own at least one side of the ledger:

- write tests first

Tests and implementation should not be invented by the same unchecked process.

If a model writes the code and the tests, it may write tests that merely confirm its own assumptions.

Humans must own at least one side of the ledger:

- write tests first
- or review AI-generated tests very closely

Tests and implementation should not be invented by the same unchecked process.

If a model writes the code and the tests, it may write tests that merely confirm its own assumptions.

Humans must own at least one side of the ledger:

- write tests first
- or review AI-generated tests very closely
- or compare against an independent implementation

Before a risky agent task:

Before a risky agent task:

```
git status  
git switch -c ai-bayesfactor-logbf  
git add .  
git commit -m "Checkpoint before AI-assisted BayesFactor extension"
```

Before a risky agent task:

```
git status
```

```
git switch -c ai-bayesfactor-logbf
```

```
git add .
```

```
git commit -m "Checkpoint before AI-assisted BayesFactor extension"
```

A branch is cheap.

Before a risky agent task:

```
git status  
git switch -c ai-bayesfactor-logbf  
git add .  
git commit -m "Checkpoint before AI-assisted BayesFactor extension"
```

A branch is cheap.

A tangled working tree is expensive. Check `git status` before committing so secrets and unrelated files stay out.

After a successful unit of work:

After a successful unit of work:

1. Review `git diff`.

After a successful unit of work:

1. Review `git diff`.
2. Update `TASKS.md` or `SCRATCHPAD.md`.

After a successful unit of work:

1. Review `git diff`.
2. Update `TASKS.md` or `SCRATCHPAD.md`.
3. Commit the working state.

After a successful unit of work:

1. Review `git diff`.
2. Update `TASKS.md` or `SCRATCHPAD.md`.
3. Commit the working state.
4. Clear or compact the context.

After a successful unit of work:

1. Review `git diff`.
2. Update `TASKS.md` or `SCRATCHPAD.md`.
3. Commit the working state.
4. Clear or compact the context.
5. Reload `AGENTS.md` and current task notes.

After a successful unit of work:

1. Review `git diff`.
2. Update `TASKS.md` or `SCRATCHPAD.md`.
3. Commit the working state.
4. Clear or compact the context.
5. Reload `AGENTS.md` and current task notes.

After a successful unit of work:

1. Review `git diff`.
2. Update `TASKS.md` or `SCRATCHPAD.md`.
3. Commit the working state.
4. Clear or compact the context.
5. Reload `AGENTS.md` and current task notes.

This protects both the codebase and the model's context.

Preparation:

Preparation:

1. Gather background knowledge before implementation.

Twelve quick tips, grouped

Preparation:

1. Gather background knowledge before implementation.
2. Distinguish problem framing from coding.

Twelve quick tips, grouped

Preparation:

1. Gather background knowledge before implementation.
2. Distinguish problem framing from coding.
3. Choose appropriate AI interaction models.

Twelve quick tips, grouped

Preparation:

1. Gather background knowledge before implementation.
2. Distinguish problem framing from coding.
3. Choose appropriate AI interaction models.
4. Think through and divide an implementation plan.

Twelve quick tips, grouped

Preparation:

1. Gather background knowledge before implementation.
2. Distinguish problem framing from coding.
3. Choose appropriate AI interaction models.
4. Think through and divide an implementation plan.

Twelve quick tips, grouped

Preparation:

1. Gather background knowledge before implementation.
2. Distinguish problem framing from coding.
3. Choose appropriate AI interaction models.
4. Think through and divide an implementation plan.

Context engineering:

Twelve quick tips, grouped

Preparation:

1. Gather background knowledge before implementation.
2. Distinguish problem framing from coding.
3. Choose appropriate AI interaction models.
4. Think through and divide an implementation plan.

Context engineering:

5. Manage context strategically.

Twelve quick tips, grouped

Preparation:

1. Gather background knowledge before implementation.
2. Distinguish problem framing from coding.
3. Choose appropriate AI interaction models.
4. Think through and divide an implementation plan.

Context engineering:

5. Manage context strategically.
6. Meticulously verify AI assistance for testing.

Testing and validation:

Testing and validation:

7. Use tests to specify and constrain AI-generated code.

Testing and validation:

7. Use tests to specify and constrain AI-generated code.
8. Leverage AI to audit and expand test coverage.

Testing and validation:

7. Use tests to specify and constrain AI-generated code.
8. Leverage AI to audit and expand test coverage.
9. Monitor progress and know when to restart.

Testing and validation:

7. Use tests to specify and constrain AI-generated code.
8. Leverage AI to audit and expand test coverage.
9. Monitor progress and know when to restart.

Testing and validation:

7. Use tests to specify and constrain AI-generated code.
8. Leverage AI to audit and expand test coverage.
9. Monitor progress and know when to restart.

Code quality:

Testing and validation:

7. Use tests to specify and constrain AI-generated code.
8. Leverage AI to audit and expand test coverage.
9. Monitor progress and know when to restart.

Code quality:

10. Critically review generated code.

Testing and validation:

7. Use tests to specify and constrain AI-generated code.
8. Leverage AI to audit and expand test coverage.
9. Monitor progress and know when to restart.

Code quality:

10. Critically review generated code.
11. Refine code incrementally with focused objectives.

Testing and validation:

7. Use tests to specify and constrain AI-generated code.
8. Leverage AI to audit and expand test coverage.
9. Monitor progress and know when to restart.

Code quality:

10. Critically review generated code.
11. Refine code incrementally with focused objectives.
12. Document your code for reuse and publication.

When AI-generated code contributes to a scientific result, the scientist remains responsible.

When AI-generated code contributes to a scientific result, the scientist remains responsible.

“AI wrote it” is not a defense.

When AI-generated code contributes to a scientific result, the scientist remains responsible.

“AI wrote it” is not a defense.

You must be able to:

When AI-generated code contributes to a scientific result, the scientist remains responsible.

“AI wrote it” is not a defense.

You must be able to:

- explain the method

When AI-generated code contributes to a scientific result, the scientist remains responsible.

“AI wrote it” is not a defense.

You must be able to:

- explain the method
- justify the implementation

When AI-generated code contributes to a scientific result, the scientist remains responsible.

“AI wrote it” is not a defense.

You must be able to:

- explain the method
- justify the implementation
- reproduce the environment

When AI-generated code contributes to a scientific result, the scientist remains responsible.

“AI wrote it” is not a defense.

You must be able to:

- explain the method
- justify the implementation
- reproduce the environment
- document the workflow

When AI-generated code contributes to a scientific result, the scientist remains responsible.

“AI wrote it” is not a defense.

You must be able to:

- explain the method
- justify the implementation
- reproduce the environment
- document the workflow
- validate the output

When AI-generated code contributes to a scientific result, the scientist remains responsible.

“AI wrote it” is not a defense.

You must be able to:

- explain the method
- justify the implementation
- reproduce the environment
- document the workflow
- validate the output

When AI-generated code contributes to a scientific result, the scientist remains responsible.

“AI wrote it” is not a defense.

You must be able to:

- explain the method
- justify the implementation
- reproduce the environment
- document the workflow
- validate the output

The tool can assist.

When AI-generated code contributes to a scientific result, the scientist remains responsible.

“AI wrote it” is not a defense.

You must be able to:

- explain the method
- justify the implementation
- reproduce the environment
- document the workflow
- validate the output

The tool can assist.

It cannot be accountable.

Do not submit sensitive material to cloud AI tools unless you have explicit approval.

Do not submit sensitive material to cloud AI tools unless you have explicit approval.

Examples:

Do not submit sensitive material to cloud AI tools unless you have explicit approval.

Examples:

- patient information

Do not submit sensitive material to cloud AI tools unless you have explicit approval.

Examples:

- patient information
- identifiable participant data

Do not submit sensitive material to cloud AI tools unless you have explicit approval.

Examples:

- patient information
- identifiable participant data
- embargoed results

Do not submit sensitive material to cloud AI tools unless you have explicit approval.

Examples:

- patient information
- identifiable participant data
- embargoed results
- proprietary code

Do not submit sensitive material to cloud AI tools unless you have explicit approval.

Examples:

- patient information
- identifiable participant data
- embargoed results
- proprietary code
- private API keys

Do not submit sensitive material to cloud AI tools unless you have explicit approval.

Examples:

- patient information
- identifiable participant data
- embargoed results
- proprietary code
- private API keys

Do not submit sensitive material to cloud AI tools unless you have explicit approval.

Examples:

- patient information
- identifiable participant data
- embargoed results
- proprietary code
- private API keys

When in doubt, stop and ask.

The working rule

Use AI as a fast collaborator whose work must be checked.

The working rule

Use AI as a fast collaborator whose work must be checked.

Not as:

The working rule

Use AI as a fast collaborator whose work must be checked.

Not as:

- an authority

The working rule

Use AI as a fast collaborator whose work must be checked.

Not as:

- an authority
- an oracle

Use AI as a fast collaborator whose work must be checked.

Not as:

- an authority
- an oracle
- a replacement for tests

Use AI as a fast collaborator whose work must be checked.

Not as:

- an authority
- an oracle
- a replacement for tests
- a replacement for domain knowledge

Use AI as a fast collaborator whose work must be checked.

Not as:

- an authority
- an oracle
- a replacement for tests
- a replacement for domain knowledge
- a reason to skip review

Use AI as a fast collaborator whose work must be checked.

Not as:

- an authority
- an oracle
- a replacement for tests
- a replacement for domain knowledge
- a reason to skip review

The working rule

Use AI as a fast collaborator whose work must be checked.

Not as:

- an authority
- an oracle
- a replacement for tests
- a replacement for domain knowledge
- a reason to skip review

The faster the draft appears, the more disciplined the review must be.